Ministerul Educației și Cercetării al Republicii Moldova

Universitatea Tehnică a Moldovei

Facultatea Calculatoare, Informatică și Microelectronică

Program: Software Engineering

Operating Systems

Laboratory Work No. 4

The xv6 Shell from the Outside

Track B: Build Your Own Small OS, xv6

Elaborated by:
st. gr. FAF-243
Veacheslav Morari

Verified by:
Patricia Reitman

Chișinău – 2026

Contents

Introduction **2**

1. Fix sleep for the current xv6 **2**

2. bash versus the xv6 shell **3**

2.1. Observations 5

3. Who expands *? A program that prints its argv **6**

3.1. Observations 7

4. Read the shell **7**

4.1. Observations 9

Conclusion **9**

Introduction

The purpose of this laboratory work is to study the shell of xv6 from the outside and to compare it with bash. The whole user interface of xv6 is one program, `user/sh.c`, of about 500 lines, so it is a good place to see what a shell must do and what bash adds on top. A small program that prints its arguments is used to show which part of the system expands `*` and removes quotes.

The work was done on Ubuntu 26.04.1 LTS with xv6-riscv in `~/xv6-riscv` (Git branch `lab`, base commit `06aad25`). xv6 was started with `make qemu CPUS=1` and left with Ctrl+A, then X.

1. Fix sleep for the current xv6

In August 2025 the xv6 authors renamed the system call `sleep()` to `pause()`, so the `sleep` program has to be checked on the current tree. The command `grep -n "pause\|sleep" user/user.h` printed only `25:int pause(int);`, which means that this tree declares `pause` and the call in `sleep.c` must be changed.

```
veacheslav@os-lab:~/xv6-riscv$  grep -n "pause\|sleep" user/user.h
25:int pause(int);
```

Figure 1.1 – The grep result in user.h

The program `user/sleep.c` checks that exactly one argument was given, prints `usage: sleep <ticks>` to standard error and exits with code 1 otherwise, and then calls `pause(atoi(argv[1]))`. Here `argc` is the number of arguments, `argv[1]` is the text typed by the user, `atoi()` turns it into an integer, and `pause()` suspends the process for that number of ticks. The line `$U/_sleep\` was added to the `UPROGS` list of the `Makefile`, next to the other user programs; it starts with a tab and nothing follows the backslash.

```
veacheslav@os-lab:~/xv6-riscv$  cat user/sleep.c
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
    if(argc != 2){
        fprintf(2, "usage: sleep <ticks>\n");
        exit(1);
    }
    pause(atoi(argv[1]));
    exit(0);
}
```

Figure 1.2 – The updated sleep.c

After `make qemu CPUS=1` the shell started and `sleep 20` finished silently after a short pause, while `sleep` without an argument printed the usage message.

```
veacheslav@os-lab:~/xv6-riscv$  make qemu CPUS=1
xv6 kernel is booting
init: starting sh
$ sleep 20
$ sleep
usage: sleep <ticks>
$
```

Figure 1.3 – sleep running in xv6, with and without an argument

2. bash versus the xv6 shell

Every command was run twice: in bash in an empty test folder containing a file `README` (`mkdir ~/shtest && cd ~/shtest && touch README`) and at the xv6 prompt.

```
veacheslav@os-lab:~/shtest$  mkdir -p ~/shtest && cd ~/shtest && touch README
veacheslav@os-lab:~/shtest$  echo hello > f ; cat < f
hello
veacheslav@os-lab:~/shtest$  ls | grep READ
README
veacheslav@os-lab:~/shtest$  (echo a; echo b) | wc
      2       2       4
veacheslav@os-lab:~/shtest$  echo one ; echo two
one
two
veacheslav@os-lab:~/shtest$  echo bg &
bg
[1]+  Done                    echo bg
veacheslav@os-lab:~/shtest$  ls *
README  f
veacheslav@os-lab:~/shtest$  echo $HOME
/home/veacheslav
veacheslav@os-lab:~/shtest$  echo "a b"
a b
veacheslav@os-lab:~/shtest$  mkdir d ; cd d ; ls ..
README  d  f
```

Figure 2.1 – The commands run in bash

```
$ echo hello > f ; cat < f
hello
$ ls | grep READ
README          2 2 2441
$ (echo a; echo b) | wc
2 2 4
$ echo one ; echo two
one
two
$ echo bg &
bg
$ ls *
ls: cannot open *
$ echo $HOME
$HOME
$ echo "a b"
"a b"
$ mkdir d
$ cd d
$ ls ..
exec ls failed
```

Figure 2.2 – The same commands run in xv6

Command (tests)	bash	xv6	Why
`echo hello > f ; cat < f` (redirection)	`hello`	`hello`	Both shells open the file and connect it to the standard input or output of the command.
`ls | grep READ` (pipe)	`README`	`README 2 2 2441`	The pipe works in both. The xv6 `ls` prints the type, inode number and size next to each name, so `grep` gets a longer line.
`(echo a; echo b) | wc` (grouping, list)	`2 2 4`	`2 2 4`	Both run the group and count 2 lines, 2 words and 4 bytes.
`echo one ; echo two` (sequence)	`one`, `two`	`one`, `two`	The `;` operator is supported by both.
`echo bg &` (background)	`bg`, then `Done`	`bg`	Both start the command without waiting. bash also tracks jobs and reports `Done`; xv6 does not.
`ls *` (globbing)	`README f`	`ls: cannot open *`	bash expands `*` to file names. xv6 passes the character `*` to `ls`, which looks for a file with that name.
`echo $HOME` (variables)	`/home/veacheslav`	`$HOME`	bash substitutes the variable. xv6 has no such substitution and prints the text.
`echo "a b"` (quotes)	`a b`	`"a b"`	bash removes the quotes and keeps one argument. xv6 keeps the quote characters.
`mkdir d ; cd d ; ls ..` (PATH)	`README d f`	`exec ls failed`	bash finds `ls` through `PATH`. xv6 looks for the file `ls` in the current directory, where it does not exist.
arrow up, Tab, Ctrl+R (history, completion)	work	nothing happens	The xv6 shell only reads a line from the console and has no history or completion code.

```
$ mkdir d
$ cd d
$ ls ..
exec ls failed
```

Figure 2.3 – ls failing after cd d

2.1. Observations

(1) Both shells have redirection with `<` and `>`, pipes, grouping with parentheses, the sequence operator `;` and the background operator `&` (in a simpler form in xv6). Only bash has file name expansion with `*`, variable substitution, handling of quotes, command history with the arrow keys, Tab completion and the Ctrl+R search.

(2) After `cd d` the command `ls` stops working because xv6 has no `PATH`. The shell takes the word `ls` as the name of a file and opens it in the current directory, but all programs of xv6 live in the root directory of its file system, and `d` does not contain `ls`. The way out is to give the full name of the program, `/ls ..`, which runs the program from the root directory.

(3) In daily work I would miss command history and Tab completion most, because they save the most typing. The easiest feature to add is the handling of quotes: the shell already splits the line into words in one function, so it would be enough to treat text between two quote characters as one word. Variable substitution is harder, because xv6 has no environment variables at all.

3. Who expands *? A program that prints its argv

The program `user/args.c` prints every element of `argv`, and the line `$U/_args\` was added to `UPROGS`. The same program was written for Linux as `args_linux.c` with `<stdio.h>` and `return 0`, and compiled with `gcc -o args args_linux.c`.

```
veacheslav@os-lab:~/xv6-riscv$  cat user/args.c
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
    int i;

    for(i = 0; i < argc; i++){
        printf("argv[%d] = %s\n", i, argv[i]);
    }

    exit(0);
}
veacheslav@os-lab:~/xv6-riscv$  grep -n "_sleep\|_args" Makefile
        $U/_sleep\
        $U/_args\
```

Figure 3.1 – args.c and the UPROGS line

In xv6, `args one two three` printed four arguments, and `args "x y"` printed three: `args`, `"x` and `y"`.

```
$ args one two three
argv[0] = args
argv[1] = one
argv[2] = two
argv[3] = three
$ args "x y"
argv[0] = args
argv[1] = "x
argv[2] = y"
```

Figure 3.2 – args output in xv6

In bash the same lines were run as `./args` in a folder with one `.c` file. The spaces between `one`, `two` and `three` gave no empty arguments, `*.c` became `args_linux.c`, `"x y"` stayed one argument without quotes, `$HOME` became `/home/veacheslav`, and `'single quoted'` was one argument without quotes.

```
veacheslav@os-lab:~/shtest$  ./args one             two        three
argv[0] = ./args
argv[1] = one
argv[2] = two
argv[3] = three
veacheslav@os-lab:~/shtest$  ./args *.c "x y"
argv[0] = ./args
argv[1] = args_linux.c
argv[2] = x y
veacheslav@os-lab:~/shtest$  ./args $HOME
argv[0] = ./args
argv[1] = /home/veacheslav
veacheslav@os-lab:~/shtest$  ./args 'single quoted'
argv[0] = ./args
argv[1] = single quoted
```

Figure 3.3 – args_linux output in bash

3.1. Observations

(1) In bash, `./args *.c "x y"` gave three arguments: `./args`, `args_linux.c` and `x y`. In xv6 the shell does not expand `*` and does not group words in quotes, as the run of `args "x y"` showed, so there the same line would give four arguments: `args`, `*.c`, `"x` and `y"`.

(2) The shell does it. The program and the kernel only receive the finished array `argv`. bash expands `*.c`, substitutes `$HOME` and removes the quotes before it starts the program, and the xv6 shell does none of these things.

(3) The extra spaces disappeared. Both shells use whitespace only as a separator between words, so any number of spaces gives the same four arguments and no empty ones.

4. Read the shell

The file `user/sh.c` has 499 lines. The `#define` lines show the kinds of commands: `EXEC`, `REDIR`, `PIPE`, `LIST` and `BACK`, together with `MAXARGS` (10), the maximum number of arguments of one command.

```
veacheslav@os-lab:~/xv6-riscv$  wc -l user/sh.c
499 user/sh.c
veacheslav@os-lab:~/xv6-riscv$  grep -n "#define" user/sh.c
8:#define EXEC  1
9:#define REDIR 2
10:#define PIPE  3
11:#define LIST  4
12:#define BACK  5
14:#define MAXARGS 10
```

Figure 4.1 – The command kinds defined in sh.c

The search for `case` shows two groups of lines. Lines 75–125 and 469–493 handle each kind of command (the first group runs the command, the second one finishes the strings of the parsed command tree), and lines 284–294 and 391–397 belong to the parser, which recognizes the special characters.

```
veacheslav@os-lab:~/xv6-riscv$  grep -n "case " user/sh.c
75:  case EXEC:
83:  case REDIR:
93:  case LIST:
101:  case PIPE:
125:  case BACK:
284:  case 0:
286:  case '|':
287:  case '(':
288:  case ')':
289:  case ';':
290:  case '&':
291:  case '<':
294:  case '>':
391:    case '<':
394:    case '>':
397:    case '+': // >>
469:  case EXEC:
475:  case REDIR:
481:  case PIPE:
487:  case LIST:
493:  case BACK:
```

Figure 4.2 – Where each kind is run

The search for `cd` found the error message `cannot cd` on line 170.

```
veacheslav@os-lab:~/xv6-riscv$  grep -n "cd" user/sh.c | head
170:        fprintf(2, "cannot cd %s\n", cmd + 3);
```

Figure 4.3 – The code that handles cd

4.1. Observations

(1) The xv6 shell knows five kinds of command. `EXEC` is a plain command with its arguments and has no special character. `REDIR` is produced by `<` and `>` (and `>>`), `PIPE` by `|`, `LIST` by `;` and `BACK` by `&`. Parentheses only group commands.

(2) The `cd` command is handled directly in `sh.c`, at the place where the message `cannot cd` is printed (line 170). It is not a separate program because `chdir` must change the current directory of the shell itself. A separate program would run in a child process and would change only its own directory, so the shell would stay where it was.

(3) The characters handled by the parser (lines 284–294) are only the end of the line, `|`, `(`, `)`, `;`, `&`, `<` and `>`, and `>>` in the redirection code. There is no code for `*`, `$` or quotes. This explains Part 2: the xv6 shell cannot expand file names, substitute variables or handle quotes, so these characters reach the program as ordinary text.

Conclusion

In this work the shell of xv6 was compared with bash. The `sleep` program was fixed for the renamed system call `pause()`, and the program `args`, written for both systems, showed exactly which arguments a program receives. The experiments confirmed that xv6 supports the main features of a command line: redirection, pipes, grouping, sequences and background commands. Bash adds file name expansion, variable substitution, quote handling, history and completion, and finds programs through `PATH`. The reading of `user/sh.c` showed why: its parser knows only five kinds of command and a few special characters, and it handles `cd` itself because a directory change must happen in the shell process. The main result is that the shell prepares the command line before the program starts, so the program only sees the finished `argv`.